

Phase 5 — Monitoring with Prometheus and Grafana

Harrington Capital plc | Infrastructure Engineer Simulation

Objective

Get all Harrington Capital hosts visible in Prometheus and build a Grafana dashboard showing live CPU, memory, disk, and network metrics. James Okafor's Week 5 deadline: all 23 managed hosts in monitoring.

By the end of this phase you will have: - node_exporter running on HC-APP01 and HC-MON01 - Windows Exporter running on HC-DC01 - All scrape targets showing `State: UP` in Prometheus - Grafana dashboard with minimum 5 panels - Dashboard JSON exported and committed

BUILD

Step 1 — Install node_exporter on Linux hosts

Run on both HC-APP01 and HC-MON01:

```
# Download latest node_exporter
wget https://github.com/prometheus/node_exporter/releases/download/v1.7.0/node_exporter-1.7.0.linux-
tar xvf node_exporter-1.7.0.linux-amd64.tar.gz
sudo cp node_exporter-1.7.0.linux-amd64/node_exporter /usr/local/bin/
sudo useradd -rs /bin/false node_exporter 2>/dev/null || true

# Create systemd service
sudo tee /etc/systemd/system/node_exporter.service > /dev/null << 'EOF'
[Unit]
Description=Node Exporter
After=network.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable node_exporter
sudo systemctl start node_exporter
sudo systemctl status node_exporter
```

Verify:

```
curl http://localhost:9100/metrics | head -20
```

Step 2 — Install Windows Exporter on HC-DC01

On HC-DC01, open PowerShell as Administrator:

```
# Download Windows Exporter MSI
$url = "https://github.com/prometheus-community/windows_exporter/releases/download/v0.24.0/windows_e
Invoke-WebRequest -Uri $url -OutFile "C:\Temp\windows_exporter.msi"

# Install
Start-Process msixexec.exe -ArgumentList "/i C:\Temp\windows_exporter.msi /quiet" -Wait

# Verify service
Get-Service windows_exporter
```

Verify (from HC-MON01):

```
curl http://<HC-DC01-IP>:9182/metrics | head -20
```

Step 3 — Configure Prometheus

On HC-MON01, update the prometheus config:

```
sudo cp harrington-infrastructure-operations/monitoring/prometheus.yml /etc/prometheus/prometheus.yml
```

Edit `/etc/prometheus/prometheus.yml` and replace the placeholder IPs with real IPs from Phase 1 terraform output.

Restart Prometheus:

```
sudo systemctl restart prometheus
sudo systemctl status prometheus
```

Step 4 — Check all targets

Open `http://<HC-MON01-IP>:9090/targets`

All jobs must show `State: UP`.

If any show `DOWN`, check: - The exporter is running on that host - Port is reachable (test with `curl`) - UFW allows the port

Take a screenshot of the /targets page with all targets UP.

Step 5 — Build the Grafana dashboard

1. Open Grafana: `http://<HC-MON01-IP>:3000`
2. Login: admin / admin (change password on first login)
3. Add Prometheus data source:
4. Configuration → Data Sources → Add data source
5. Type: Prometheus
6. URL: `http://localhost:9090`
7. Click "Save & Test" — must show green tick
8. Create a new dashboard with these 5 panels:

Panel 1 — CPU Usage % (all hosts)

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

Visualisation: Time series

Panel 2 — Memory Usage % (Linux hosts)

```
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
```

Visualisation: Gauge

Panel 3 — Disk Usage % (Linux hosts)

```
100 - ((node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes{mountpoint="/" }) * 100)
```

Visualisation: Stat

Panel 4 — Network I/O (bytes received per second)

```
rate(node_network_receive_bytes_total[5m])
```

Visualisation: Time series

Panel 5 — Up/Down status (all targets)

```
up
```

Visualisation: Stat (colour: green = 1, red = 0)

1. Name the dashboard: `Harrington Capital – Infrastructure Overview`
2. Save the dashboard.

Step 6 — Export dashboard JSON

Dashboard settings (gear icon) → JSON Model → Copy to clipboard.

Save as `/monitoring/grafana-dashboard.json` and commit:

```
git add monitoring/  
git commit -m "feat(monitors): Grafana dashboard JSON and updated prometheus.yml - Phase 5"  
git push origin main
```

VERIFY

```
# Prometheus targets all UP
curl -s http://localhost:9090/api/v1/targets | python3 -m json.tool | grep '"health"'
# All values must be "up"

# node_exporter metrics available
curl -s http://localhost:9100/metrics | grep "node_cpu_seconds_total" | head -3

# Grafana API health
curl -s http://admin:admin@localhost:3000/api/health
# Must return: {"commit":"...", "database":"ok", "version":"..."}
```

SUBMIT

#	Evidence	How to get it
1	Prometheus <code>/targets</code> screenshot	All targets showing <code>State: UP</code>
2	Grafana dashboard screenshot	Full dashboard with all 5 panels showing live data
3	<code>monitoring/grafana-dashboard.json</code>	GitHub commit URL
4	<code>prometheus.yml</code> with real IPs	GitHub commit URL
5	<code>curl /api/v1/targets</code> output	Shows all scrape jobs and their health status

Anti-fake check: Screenshots must show live timestamp in Grafana (bottom right of dashboard) and real metric values — not zero.